VajraKavach

Secure Job Search & Professional Networking Platform

Project Milestone 3 Report

CSE 345/545: Foundations to Computer Security

Team Members	Roll Number
Yash Kumar Vaibhav	MT25091
Aayush Gupta	MT25008
Ayush Jain	MT25066
Ayush Kumar	2020290

Indraprastha Institute of Information Technology, Delhi

March 31, 2026

Contents

1 Company Pages and Job Postings **3**
1.1 Overview . . . 3
1.2 Company Page Management . . . 3
1.3 Job Posting System . . . 3
1.4 API Endpoints . . . 4
1.5 Security Measures . . . 4
1.6 Screenshots . . . 5

2 Job Search and Application Workflow **7**
2.1 Overview . . . 7
2.2 Advanced Job Search . . . 7
2.3 Application Workflow . . . 7
2.4 Application Model . . . 7
2.5 Security Measures . . . 8
2.6 Screenshots . . . 8

3 Application Status Tracking **9**
3.1 Overview . . . 9
3.2 Status Workflow . . . 9
3.3 Recruiter Features . . . 9
3.4 Applicant Features . . . 9
3.5 Audit Trail . . . 9
3.6 Screenshots . . . 10

4 Encrypted One-to-One and Group Messaging **11**
4.1 Overview . . . 11
4.2 Encryption Architecture . . . 11
4.3 One-to-One Messaging . . . 11
4.4 Group Messaging . . . 11
4.5 Client-Side Key Management . . . 12
4.6 Server-Side Validation . . . 12
4.7 Group Management . . . 12
4.8 Security Summary . . . 12
4.9 Screenshots . . . 13

5 Server-Side Encrypted Announcements **15**
5.1 Overview . . . 15
5.2 Announcement Flow . . . 15
5.3 Security Measures . . . 15

6 Initial Admin Logging **16**
6.1 Overview . . . 16
6.2 Hash-Chained Audit Logs . . . 16
6.3 Logged Actions . . . 16
6.4 Blockchain Layer . . . 17
6.5 Integrity Verification . . . 17

6.6 Screenshots 17

7 Bonus Features **19**
7.1 Blockchain-Based Tamper-Evident Logging (+6%) 19
7.2 Resume Parsing and Intelligent Matching (+2%) 19

8 User Reporting System **20**
8.1 Overview 20
8.2 Report Categories 20
8.3 Admin Review Workflow 20

9 Technology Stack Compliance **21**

10 Summary **22**

Company Pages and Job Postings

Overview

VajraKavach allows recruiters and platform admins to create and manage company pages, and post job listings linked to those companies. All users can browse and search companies and jobs.

Company Page Management

1. Recruiters create a company page with name, description, location, and website URL.
2. Each company is linked to an `admin_id` (the recruiter who created it).
3. Companies can be searched by name and location (case-insensitive).
4. Only the company owner or a platform admin can update or delete a company.
5. Companies support a `is_verified` flag for admin-verified organizations.

Job Posting System

1. Recruiters create job postings linked to one of their companies.
2. Job fields include: title, description, required skills, location, salary range (optional), and application deadline.
3. Jobs support enums for consistency:
   - **Location Type:** Remote, On-site, Hybrid.
   - **Job Type:** Full-time, Part-time, Internship, Contract.
4. Jobs have an `is_active` flag allowing recruiters or admins to delist postings.
5. Applicant count is tracked and displayed per job listing.

API Endpoints

Method	Endpoint	Access
POST	`/api/companies/`	Recruiter, Admin
GET	`/api/companies/search`	Authenticated
GET	`/api/companies/my`	Recruiter
PUT	`/api/companies/{id}`	Owner, Admin
DELETE	`/api/companies/{id}`	Owner, Admin
POST	`/api/jobs/`	Recruiter, Admin
GET	`/api/jobs/search`	Authenticated
GET	`/api/jobs/my`	Recruiter
PUT	`/api/jobs/{id}`	Owner, Admin
DELETE	`/api/jobs/{id}`	Owner, Admin

Table 1: Company and Job API Endpoints

Security Measures

- **Role-Based Access Control:** Only recruiters and platform admins can create companies and post jobs.
- **Ownership Verification:** Modifications require company ownership or admin privileges.
- **Input Validation:** Pydantic schemas enforce valid data types, URL formats, and salary ranges.
- **Audit Logging:** All company and job CRUD operations are logged in the tamper-evident audit trail.

Screenshots

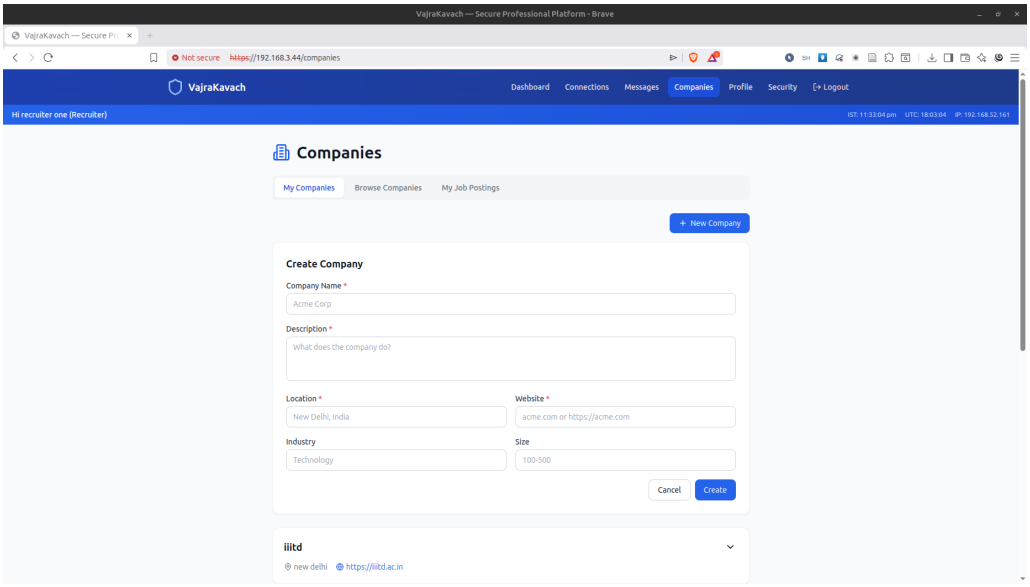


Figure 1: Company page creation form for recruiters

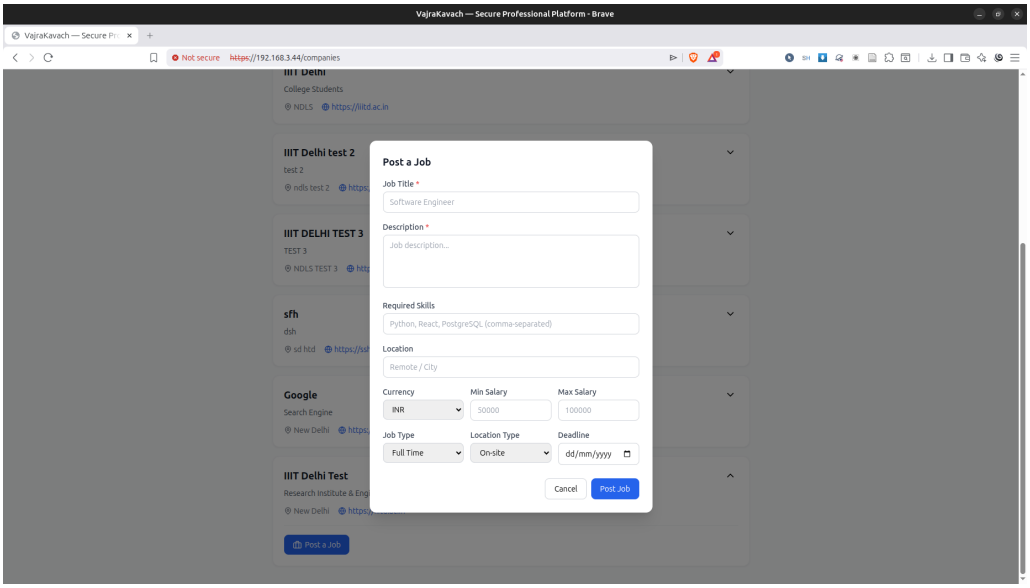


Figure 2: Job posting form with title, description, skills, location type, and salary range

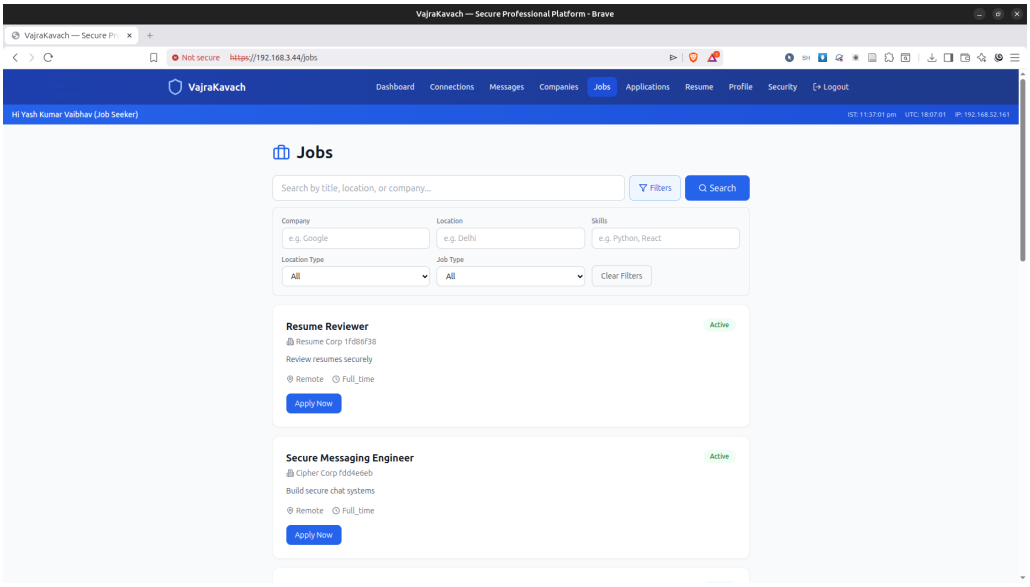


Figure 3: Job search page with keyword, location, job type, and skill filters

Job Search and Application Workflow

Overview

Job seekers can search for jobs using advanced filters, view job details, and submit applications with an optional resume and cover note. The application system prevents duplicate submissions and enforces role-based access.

Advanced Job Search

The search system supports multiple simultaneous filters:

- **Keyword search:** Matches against job title and description.
- **Company filter:** Filter by company name.
- **Location filter:** Case-insensitive location matching.
- **Location type:** Remote, On-site, or Hybrid.
- **Job type:** Full-time, Part-time, Internship, Contract.
- **Skills filter:** Comma-separated skill tags with case-insensitive substring matching.
- **Pagination:** Default page size of 20, maximum 100 results per page.

Application Workflow

1. Job seeker browses or searches available jobs.
2. Selects a job and clicks "Apply."
3. Optionally attaches a previously uploaded resume and writes a cover note.
4. System validates no duplicate application exists for the same job.
5. Application is created with status `APPLIED`.
6. Recruiter reviews applications and updates status through the workflow.

Application Model

Field	Description
`id`	UUID primary key
`job_id`	Foreign key to the job listing
`user_id`	Foreign key to the applicant
`resume_id`	Optional link to an encrypted resume
`cover_note`	Optional cover letter text
`status`	Enum: applied, reviewed, interviewed, rejected, offer
`recruiter_notes`	Private notes from the recruiter
`created_at`	Submission timestamp

Table 2: Application Data Model

Security Measures

- **Duplicate Prevention:** Server rejects multiple applications to the same job from the same user.
- **Authorization:** Only the applicant can withdraw; only the recruiter or admin can update status.
- **Audit Trail:** Every application submission and status change is logged.

Screenshots

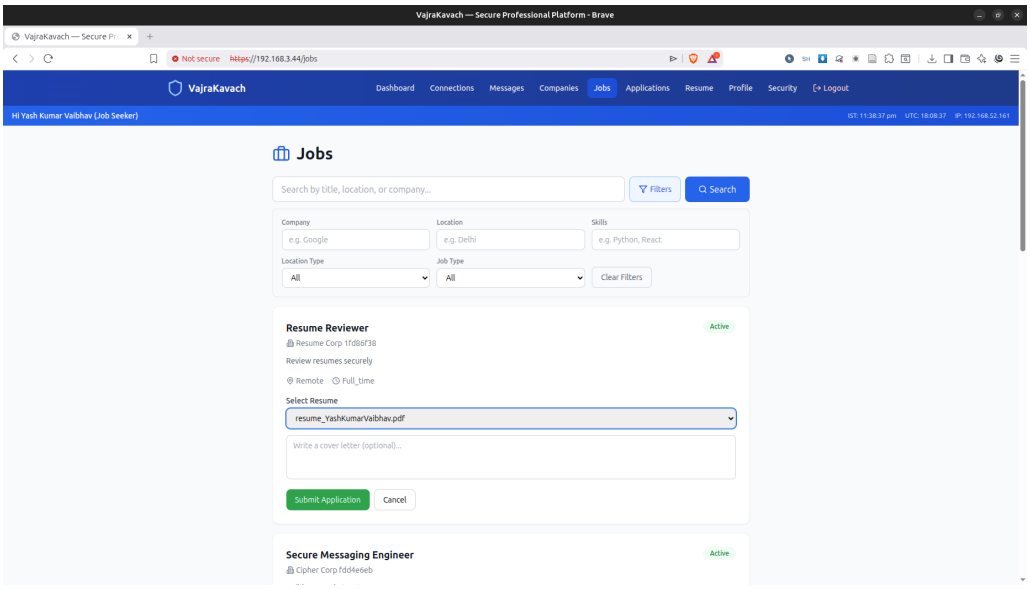


Figure 4: Job application form with resume selection and cover note

Application Status Tracking

Overview

Applications follow a defined lifecycle managed by recruiters. Both applicants and recruiters have dedicated views to track application progress.

Status Workflow

1. **Applied** — Initial state when a job seeker submits an application.
2. **Reviewed** — Recruiter has reviewed the application.
3. **Interviewed** — Candidate has been scheduled or completed an interview.
4. **Offer** — Job offer has been extended to the candidate.
5. **Rejected** — Application has been declined.

Recruiter Features

- View all applications for a specific job with applicant details.
- Update application status through the defined workflow states.
- Add private recruiter notes during status updates.
- View applicant profiles and download resumes (TOTP-gated).

Applicant Features

- View all submitted applications with current status.
- Color-coded status badges for quick visual identification.
- Withdraw pending applications before review.
- View job and company details for each application.

Audit Trail

Each status update creates an audit log entry containing:

```
{
  "action": "application.status_update",
  "resource": "application",
  "details": {
    "old_status": "applied",
    "new_status": "reviewed"
  },
  "ip_address": "<client_ip>"
}
```

Screenshots

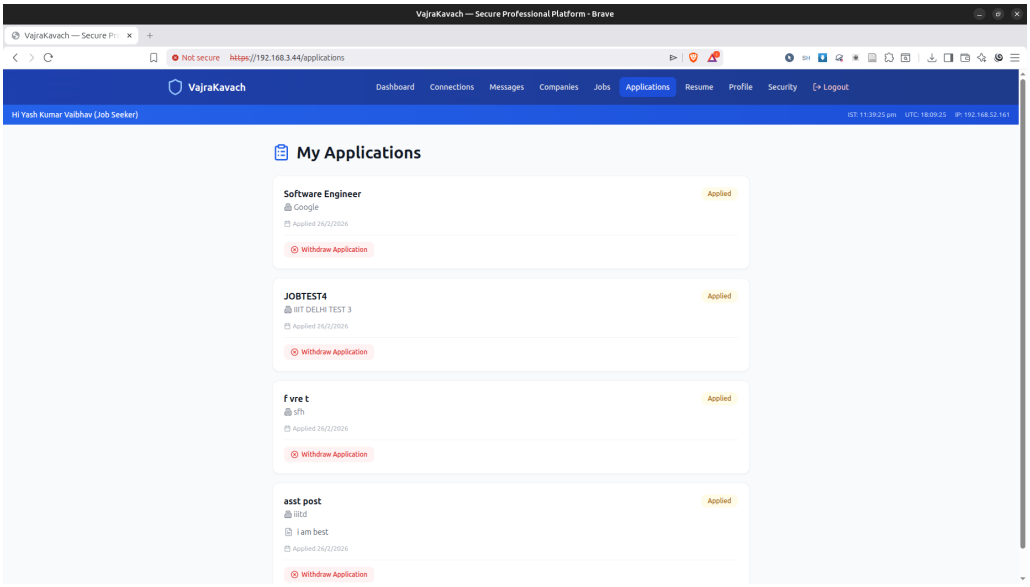


Figure 5: Applicant's view showing application status with color-coded badges

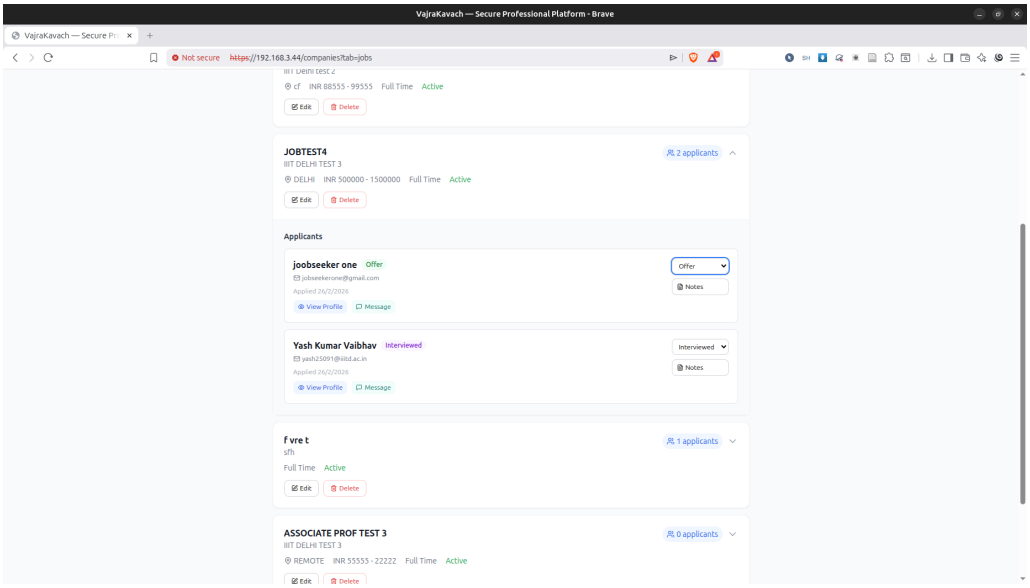


Figure 6: Recruiter's view of applications for a job with status update controls

Encrypted One-to-One and Group Messaging

Overview

VajraKavach implements a zero-knowledge messaging system where the server stores **only ciphertext**. All message encryption and decryption happens client-side using the Web Crypto API. The system supports both one-to-one and group conversations with end-to-end encryption (E2EE).

Encryption Architecture

The messaging system uses a **hybrid encryption** scheme combining RSA and AES:

1. A random **AES-256** session key is generated for each message.
2. The message plaintext is encrypted with **AES-GCM** (256-bit key, 12-byte IV, 128-bit auth tag).
3. The AES session key is encrypted with the recipient's **RSA-OAEP** public key (2048-bit).
4. The encrypted payload is assembled as:

```
[2-byte key_length][RSA-encrypted AES key][12-byte IV][AES-GCM
    ciphertext + tag]
```

The entire payload is Base64-encoded for transmission and storage.

One-to-One Messaging

For direct messages, the sender encrypts the message twice — once with the recipient's public key and once with their own — so both parties can decrypt:

```
E2E:{"s":"<encrypted_for_sender>","r":"<encrypted_for_recipient
    >"}
```

Group Messaging

For group conversations, each member receives an individually encrypted copy of the AES session key:

```
E2EG:{"<user_id_1>":"<encrypted_blob>",
      "<user_id_2>":"<encrypted_blob>", ...}
```

Each participant's blob contains the AES key encrypted with their RSA public key, ensuring only group members can decrypt.

Client-Side Key Management

- RSA-2048 key pairs are generated during user registration.
- Public keys are stored on the server for key exchange.
- Private keys are encrypted with **PBKDF2** (250,000 iterations, SHA-256) derived from the user's password and stored locally in **IndexedDB**.
- During a session, the decrypted private key is held in `sessionStorage` and cleared on tab close.

Server-Side Validation

The backend enforces that only encrypted messages are accepted:

- Rejects any message not prefixed with `E2E:` or `E2EG:`.
- Validates the JSON structure of the encrypted payload.
- Verifies the Base64-encoded ciphertext contains a valid RSA-OAEP + AES-GCM hybrid structure.
- **Never stores or processes plaintext message content.**

Group Management

- Any authenticated user can create a group conversation with selected members.
- Existing members can add new members to the group.
- The group creator or the member themselves can remove members.
- Group membership changes are validated server-side.

Security Summary

Property	Implementation
Confidentiality	RSA-OAEP (2048-bit) + AES-256-GCM hybrid encryption
Integrity	AES-GCM 128-bit authentication tag
Zero Knowledge	Server stores only ciphertext; no plaintext access
Key Exchange	RSA public keys exchanged via authenticated API
Key Storage	PBKDF2-encrypted private keys in IndexedDB
Forward Secrecy	Per-message random AES session keys

Table 3: E2EE Security Properties

Screenshots

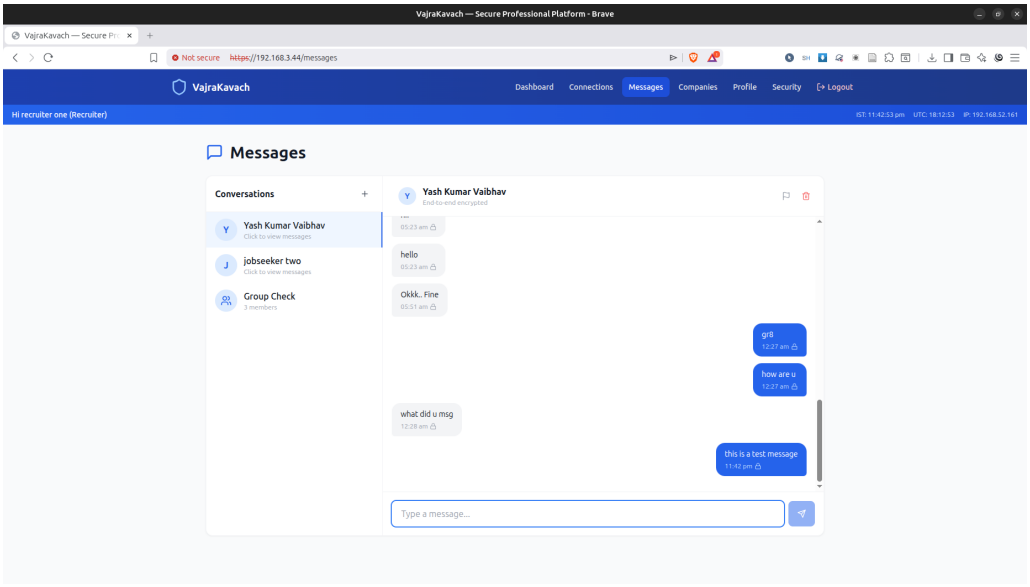


Figure 7: One-to-one encrypted conversation showing lock icon and decrypted messages

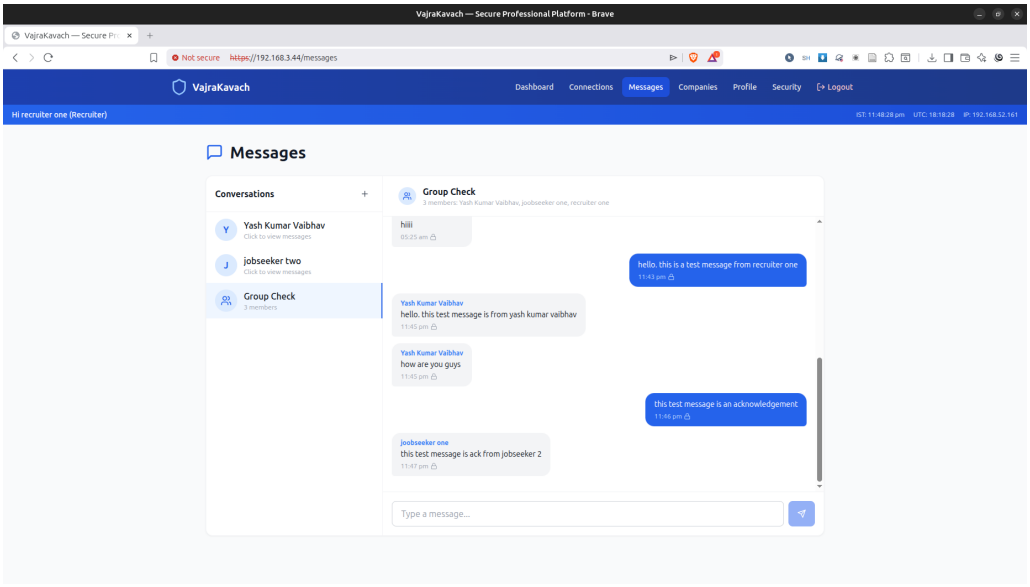


Figure 8: Group messaging with multiple participants and E2EE indicator

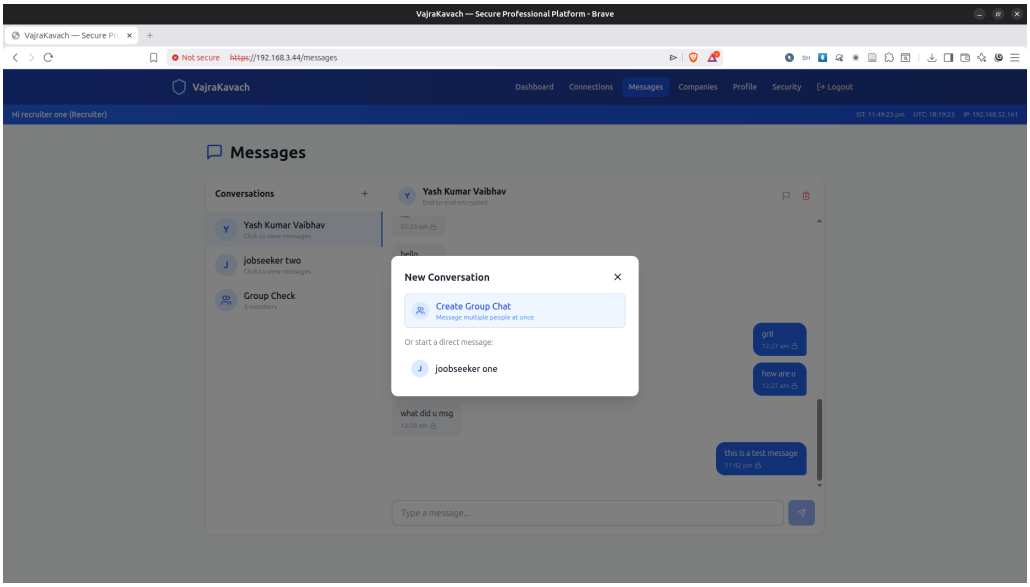


Figure 9: Conversations list showing one-to-one and group chats

Server-Side Encrypted Announcements

Overview

Platform admins can broadcast announcements to all users. Announcements use server-side encryption with **Fernet (AES-128-CBC + HMAC-SHA256)** for at-rest protection, distinct from E2EE messaging.

Announcement Flow

1. Platform admin composes an announcement with title and content.
2. Both title and content are encrypted server-side using Fernet before storage.
3. All authenticated users can retrieve announcements; the server decrypts on-the-fly.
4. Announcements support soft-delete via an `is_active` flag.

Security Measures

- Only platform admins can create or delete announcements.
- Encryption key is stored in environment configuration, never in source code.
- Decryption errors are handled gracefully without exposing internals.

Initial Admin Logging

Overview

VajraKavach implements a two-tier tamper-evident logging system: **hash-chained audit logs** and a **blockchain layer** with proof-of-work. All critical platform actions are recorded and verifiable.

Hash-Chained Audit Logs

Every critical action on the platform generates an audit log entry with the following structure:

Field	Description
`action`	Action identifier (e.g., `user.login`, `job.create`)
`resource`	Affected entity type (user, job, application, etc.)
`resource_id`	UUID of the affected entity
`details`	JSON with contextual information
`ip_address`	Client IP address (supports IPv6)
`entry_hash`	SHA-256 hash of this entry + previous hash
`previous_hash`	SHA-256 hash of the preceding entry

Table 4: Audit Log Entry Structure

Hash Chain Construction:

```
entry_hash = SHA-256(
  previous_hash | action | user_id | resource |
  resource_id | details | ip_address | timestamp
)
```

The first entry uses a genesis hash of 64 zeros. Modifying any entry breaks all subsequent hashes in the chain.

Logged Actions

The following categories of actions are logged:

- **Authentication:** Login, registration, password changes.
- **Companies:** Create, update, delete operations.
- **Jobs:** Create, update, delete, delist operations.
- **Applications:** Submissions and all status transitions.
- **Messages:** Send events (content not logged — only metadata).
- **Admin Actions:** User suspensions, promotions, deletions.
- **Blockchain:** Block mining operations.

Blockchain Layer

Audit log entries are periodically aggregated into blockchain blocks with proof-of-work:

Field	Description
`block_number`	Sequential block index
`previous_block_hash`	SHA-256 of the previous block
`merkle_root`	Merkle tree root of constituent entry hashes
`nonce`	Proof-of-work nonce
`block_hash`	SHA-256 hash with 2 leading zeros (difficulty target)
`entry_count`	Number of audit entries in the block
`entry_ids`	Comma-separated audit log entry IDs

Table 5: Blockchain Block Structure

Merkle Tree: Entry hashes are paired and hashed recursively until a single root remains, cryptographically committing to all entries in the block.

Proof-of-Work: The block hash must start with 2 leading zeros (difficulty = 2). A nonce is incremented until a valid hash is found.

Integrity Verification

Two verification endpoints are provided:

- **Audit Log Verification** (`/api/admin/audit-logs/verify-integrity`): Recomputes each entry's hash and validates the chain linkage.
- **Blockchain Verification** (`/api/admin/blockchain/verify`): Validates block hashes, proof-of-work, Merkle roots, and sequential block numbering.

Screenshots

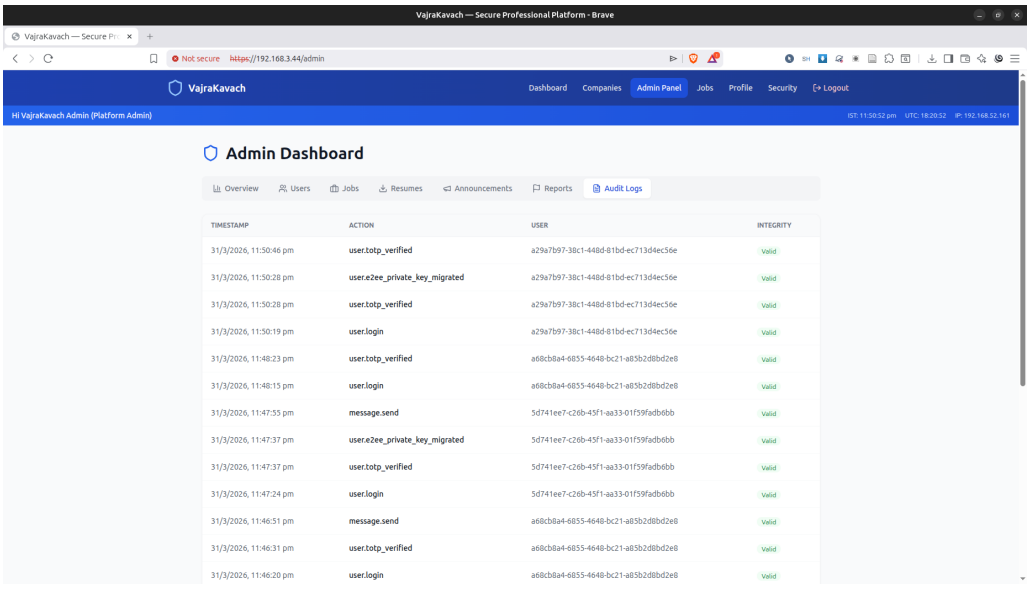


Figure 10: Admin audit logs view showing hash-chained entries with action details

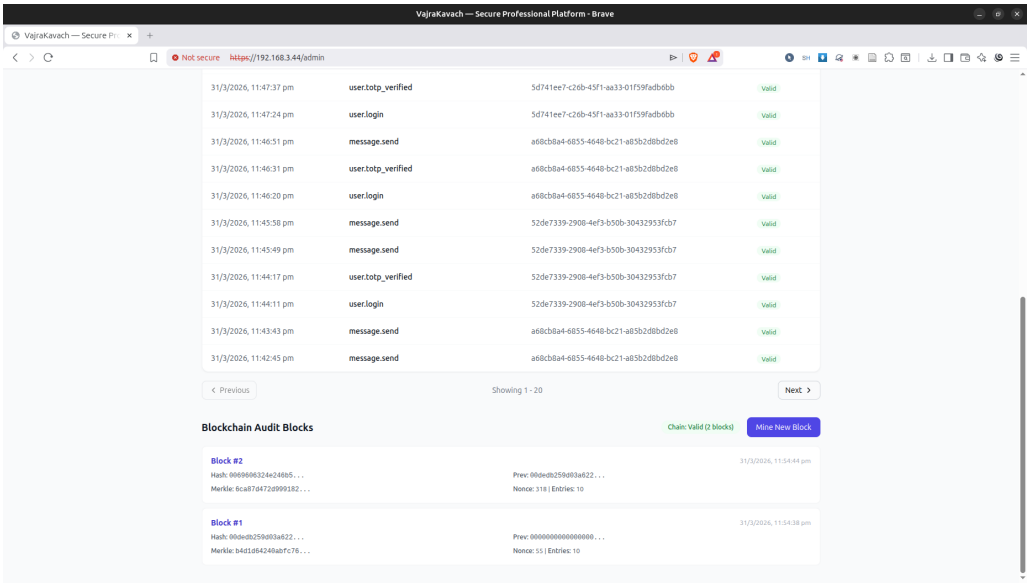


Figure 11: Blockchain blocks view showing block hashes, Merkle roots, and entry counts

Bonus Features

Blockchain-Based Tamper-Evident Logging (+6%)

As described in Section 6, the platform implements a full blockchain layer on top of hash-chained audit logs. Key properties:

- SHA-256 Merkle tree roots for cryptographic commitment to block contents.
- Proof-of-work mining with configurable difficulty.
- End-to-end verification of blockchain integrity.
- Genesis block with deterministic initial hash.

Resume Parsing and Intelligent Matching (+2%)

The platform includes automated resume parsing and job–candidate match scoring:

- **PDF Parsing:** Uploaded resumes are parsed server-side to extract text and identify skills using pattern matching and NLP techniques.
- **Skill Extraction:** Detected skills are stored alongside the resume for quick matching.
- **Match Scoring:** When a candidate applies to a job, the system computes a match score by comparing extracted resume skills against the job's required skills.
- **Recruiter View:** Match scores are displayed to recruiters in the application review interface.

User Reporting System

Overview

Users can report other users for policy violations. Reports are reviewed by admins through the admin dashboard.

Report Categories

- **Spam** — Unsolicited or irrelevant content.
- **Harassment** — Abusive or threatening behavior.
- **Fraud** — Deceptive job postings or profiles.
- **Inappropriate Content** — Violation of community guidelines.
- **Other** — General policy violations.

Admin Review Workflow

1. Reports appear in the admin dashboard under the Reports tab.
2. Admins can review report details (reporter, reported user, category, description).
3. Actions: Mark as Reviewed, Resolve, or Dismiss.
4. All report actions are logged in the audit trail.

Technology Stack Compliance

Requirement	Technology	Status
Operating System	Ubuntu (VM)	✓
Database	PostgreSQL	✓
Web Server	Nginx (reverse proxy + TLS)	✓
Backend	Python (FastAPI)	✓
Frontend	React (Vite)	✓
Password Hashing	Argon2id (passlib)	✓
TOTP	pyotp + qrcode	✓
E2EE	RSA-OAEP + AES-256-GCM (Web Crypto API)	✓
Resume Encryption	Fernet (AES-128-CBC + HMAC-SHA256)	✓
Audit Logging	SHA-256 hash chaining	✓
Blockchain	SHA-256 proof-of-work + Merkle trees	✓
HTTPS/TLS	Self-signed certificates via Nginx	✓

Table 6: Technology Stack Compliance Matrix

Summary

Milestone 3 delivers the core job platform and secure communication features of VajraKavach:

1. **Company Pages & Job Postings** — Recruiters create company pages and post jobs with advanced attributes (location type, job type, salary range, required skills, deadlines). Role-based access control enforces ownership.
2. **Job Search & Application Workflow** — Multi-filter job search (keyword, company, location, skills, job type) with pagination. Job seekers apply with optional resume and cover note; duplicate applications are prevented.
3. **Application Status Tracking** — Five-stage status workflow (Applied → Reviewed → Interviewed → Offer / Rejected) with recruiter notes, color-coded badges, and full audit trail of every status change.
4. **End-to-End Encrypted Messaging** — Zero-knowledge architecture where the server stores only ciphertext. Hybrid RSA-OAEP + AES-256-GCM encryption for both one-to-one and group conversations. Client-side key management with PBKDF2-protected private keys in IndexedDB.
5. **Server-Side Encrypted Announcements** — Fernet-encrypted broadcast messages from platform admins with at-rest protection.
6. **Tamper-Evident Audit Logging** — SHA-256 hash-chained audit logs covering all critical platform actions, with integrity verification endpoint.
7. **Blockchain Layer (Bonus)** — Proof-of-work blockchain with Merkle tree roots aggregating audit entries into immutable, verifiable blocks.
8. **Resume Parsing & Match Scoring (Bonus)** — Automated skill extraction from uploaded PDFs with job–candidate match scoring for recruiters.

All security measures follow best practices as outlined in the course requirements. The codebase is version-controlled and ready for the Final Evaluation (PKI integration, TOTP virtual keyboard, attack defense demonstration).

Repository: https://github.com/yashkumarvaibhav/VajraKavach